INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
KULLIYYAH OF ENGINEERING

MECHATRONIC SYSTEM INTEGRATION

(MCTA 3203)

EXPERIMENT 3:

SERIAL INTERFACING WITH MICROCONTROLLER: SENSORS AND ACTUATORS

SECTION 1

GROUP 1

NO	NAME	MATRIC NUMBER	PROGRAMME
1	'AISYAH EL-'ILLIYYINA BINTI MOHD FAUZI	2319982	BMCT
2	AFIQ ASRAF BIN MOHD EZUAN'IZAM	2410541	BMCT
3	AHMAD AIMAN BIN ABDUL HALIM	2417921	BMCT
4	AFIQ HUSAINI BIN MOHAMAD IZANI	2411757	BMCT

DATE OF SUBMISSION: 25 OCTOBER 2026

ABSTRACT

This experiment focuses on the development of a motion-activated Radio Frequency Identification (RFID) access control system through the integration of sensors, actuators, and serial communication using Arduino and Python. The main objective of this experiment is to understand the interfacing of an MPU6050 sensor and RFID module with an Arduino microcontroller to develop a smart access system. The experiment consists of two parts. The first part involves detecting and tracking hand movements using the MPU6050 accelerometer, where real-time motion data is transmitted to Python through serial communication and visualized using the Matplotlib library. Specifically, telemetry data is streamed at a 20 Hz sampling rate over UART at 9600 baud to generate dynamic Cartesian trajectory plots. The second part involves integrating an RFID reader, MPU6050 sensor, servo motor, and LED indicators to develop an access control system. The system is designed to verify an authorized RFID tag and detect a circular hand gesture before granting access. Motion classification is performed in Python by computing the mean radius and cumulative angular sweep of the recorded hand path. When both conditions are satisfied, the servo motor unlocks the system, and the green LED indicates successful access. Otherwise, access is denied, and the red LED is activated. Experimental results demonstrated reliable dual-factor verification, successfully transitioning the servo actuator from 90° to 180° while maintaining stable serial transmission. Through this experiment, the application of sensor integration, real-time data processing, serial communication, and actuator control in developing an automated security system is explored.

TABLE OF CONTENT

ABSTRACT 2
TABLE OF CONTENT 3
PART 1 : REAL-TIME MOTION TRACKING USING MPU6050 SENSOR 4
- **1.0. INTRODUCTION AND THEORY** 4
- **1.1. MATERIALS AND EQUIPMENTS** 5
- **1.2. EXPERIMENTAL SETUP** 6
- **1.3. METHODOLOGY AND PROGRAMMING LOGIC** 7
- **1.4. DATA COLLECTION AND ANALYSIS** 13
- **1.5. RESULTS AND DISCUSSION** 16
- **1.6. CONCLUSION** 20

PART 2: SERVO MOTOR CONTROLLED WITH POTENTIOMETER 21
- **2.0. INTRODUCTION AND THEORY** 21
- **2.1. MATERIALS AND EQUIPMENTS** 22
- **2.2. EXPERIMENTAL SETUP** 23
- **2.3. METHODOLOGY AND PROGRAMMING LOGIC** 25
- **2.4. DATA COLLECTION AND ANALYSIS** 32
- **2.5. RESULTS AND DISCUSSION** 33
- **2.6. CONCLUSION** 35

RECOMMENDATIONS 36
REFERENCES 37
APPENDICES 38
ACKNOWLEDGEMENT 39
STUDENT'S DECLARATION 40

PART 1 : REAL-TIME MOTION TRACKING USING MPU6050 SENSOR

1.0. INTRODUCTION AND THEORY

This experiment focuses on real-time motion tracking using the MPU6050 sensor interfaced with an Arduino Uno and a Python host program. The objective is to capture hand movement as accelerometer data, display the motion path on a live X–Y graph, and automatically detect when the user performs a circular hand gesture. The motion detection developed in this part serves as the gesture verification step for the RFID access system in Part 2.

The MPU6050 is a six-axis Inertial Measurement Unit (IMU) that combines a three-axis accelerometer and a three-axis gyroscope in a single module. The accelerometer measures linear acceleration along the X, Y, and Z axes, while the gyroscope measures angular velocity about the same axes. Each reading is converted into a 16-bit digital value. At the default range of ±2 g, a reading of 16 384 corresponds to 1 g. The accelerometer also senses gravity, so a stationary sensor lying flat reads about 1 g on its Z-axis and close to zero on its X and Y axes. When a circular hand motion is performed, the X and Y acceleration values change continuously, so plotting them against each other produces a loop-shaped path.

The MPU6050 communicates with the Arduino through the I²C protocol, which uses two lines: SDA (Serial Data) connected to pin A4 and SCL (Serial Clock) connected to pin A5. The sensor uses the I²C address 0x68. When the program starts, the Arduino wakes the sensor and checks the connection. It then reads the accelerometer and gyroscope values every 50 ms, which gives a sampling rate of 20 Hz. Each reading is sent to the computer through USB serial communication at 9600 baud as one comma-separated line containing six values (ax, ay, az, gx, gy, gz).

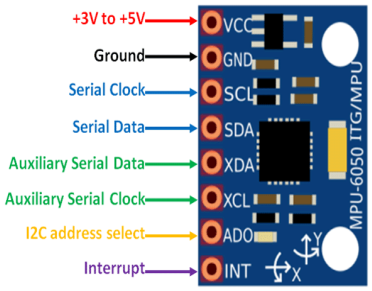
On the computer, the Python script uses the PySerial library to read the incoming data and the Matplotlib library to plot the X and Y acceleration values in real time. Only the most recent 35 readings are kept on the graph, and a simple moving average is applied to reduce noise and small hand shakes. The script then checks whether the motion is circular. A motion is accepted as circular when five conditions are all met:

1. Enough readings have been collected.
2. The movement is large enough on both axes.
3. The points surround a centre point in all four directions.
4. The distance of the points from that centre is reasonably consistent.
5. Most of the points rotate in the same direction.

When all conditions are satisfied, the graph displays “CIRCULAR MOTION DETECTED”. It is therefore expected that a smooth circular hand movement will produce a closed, round path on the graph and be detected successfully, while straight or random movements will not be recognised as circular.

1.1. MATERIALS AND EQUIPMENTS

1. Arduino Uno board
2. MPU6050 6-Axis IMU Sensor Module
    - VCC connected to 5V
    - GND connected to GND
    - SDA connected to Analog Pin A4 (I2C Data line)
    - SCL connected to Analog Pin A5 (I2C Clock line)
    - INT connected to Digital Pin D2



3. Jumper wires (male-to-female)
4. USB Cable (Type-A to Type-B)
5. Personal Computer with Arduino IDE (C++ compiler and I2C/Serial libraries)
6. Python Development Environment (PyCharm) with libraries:
    - PySerial (serial telemetry stream parsing)
    - Matplotlib (real-time 2D motion path rendering)
    - NumPy (vector calculations and gesture signal filtering)

1.2. EXPERIMENTAL SETUP

Circuit Wiring & Hardware Connection Instructions (as in Figure 1.1)**:**

1. **Arduino Uno** (acts as a base for pin connections)
2. **MPU6050 6-Axis IMU Sensor Wiring (I2C Bus Interface):**
    a. **VCC Pin** → Arduino Uno 5V Pin.
    b. **GND Pin** → Arduino Uno GND Pin.
    c. **SDA Pin (Serial Data) → Arduino Uno Analog Input Pin A4**.
    d. **SCL Pin (Serial Clock) → Arduino Analog Input Pin A5**.
    e. **INT Pin → Arduino Digital Input Pin D2**.
3. **Host USB Communications:**
    a. Connect the **Arduino Uno USB Type-B Port** to an available **USB Port** on the host PC to enable 9600 baud serial telemetry streaming.

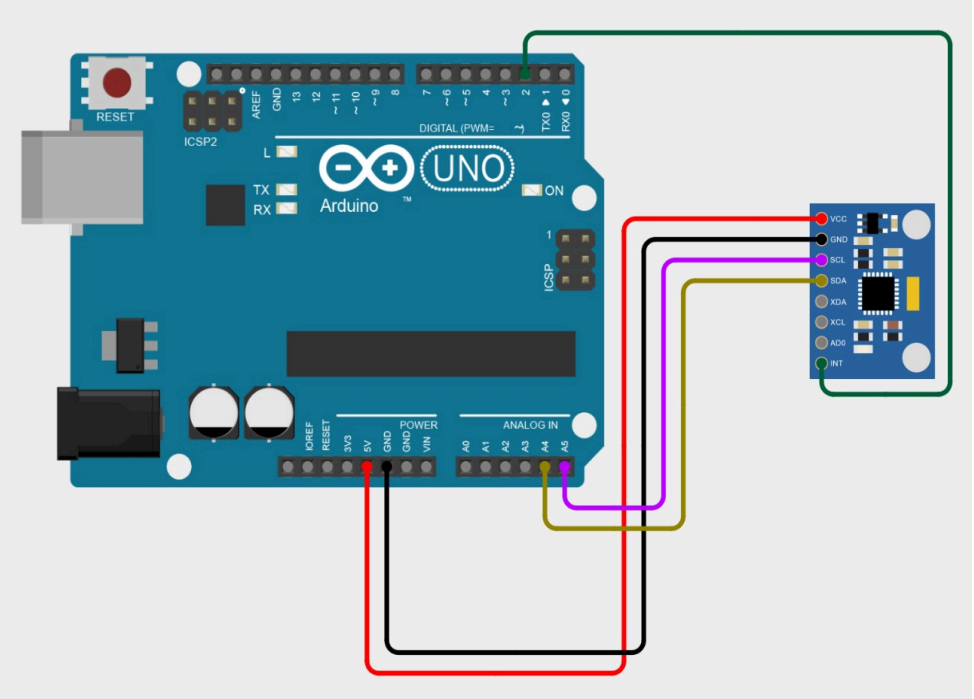


Figure 1.1: (Hardware connection between Arduino Uno and MPU6050 Sensor)

1.3. METHODOLOGY AND PROGRAMMING LOGIC

Procedural Steps:

1. Assemble the hardware components following the setup in Section 1.2.
2. Connect the Arduino Uno to the host computer using a USB cable.
3. Launch the Arduino IDE on the host PC and verify that the required libraries (Wire.h and MPU6050.h) are properly installed.
4. Open the C++ firmware sketch (W4T1.ino), select the board (Arduino Uno) and serial port (COM6), then compile and upload the code to the microcontroller.
5. Open the Arduino IDE Serial Monitor at 9600 baud to confirm that comma-separated 6-axis accelerometer and gyroscope telemetry data (ax,ay,az,gx,gy,gz) is streaming continuously.
6. Close the Serial Monitor completely to release the OS serial port handle so Python can access the communication line.
7. Open the host Python environment (PyCharm) and load the motion tracking visualization script W4T1.py).
8. Execute the Python script to open the PySerial connection and initialize the dynamic Matplotlib X-Y trajectory plotting window.
9. Perform a circular motion gesture using the MPU6050 sensor in hand.
10. Observe the real-time X-Y plot as Python continuously updates the motion trajectory, calculates the radius and angular sweep, and detects/verifies the circular motion gesture on the graph.
11. Safely terminate program execution by pressing Ctrl + C in the terminal to close the serial connection cleanly and exit.

Logic Flow Diagrams (Figure 1.2):

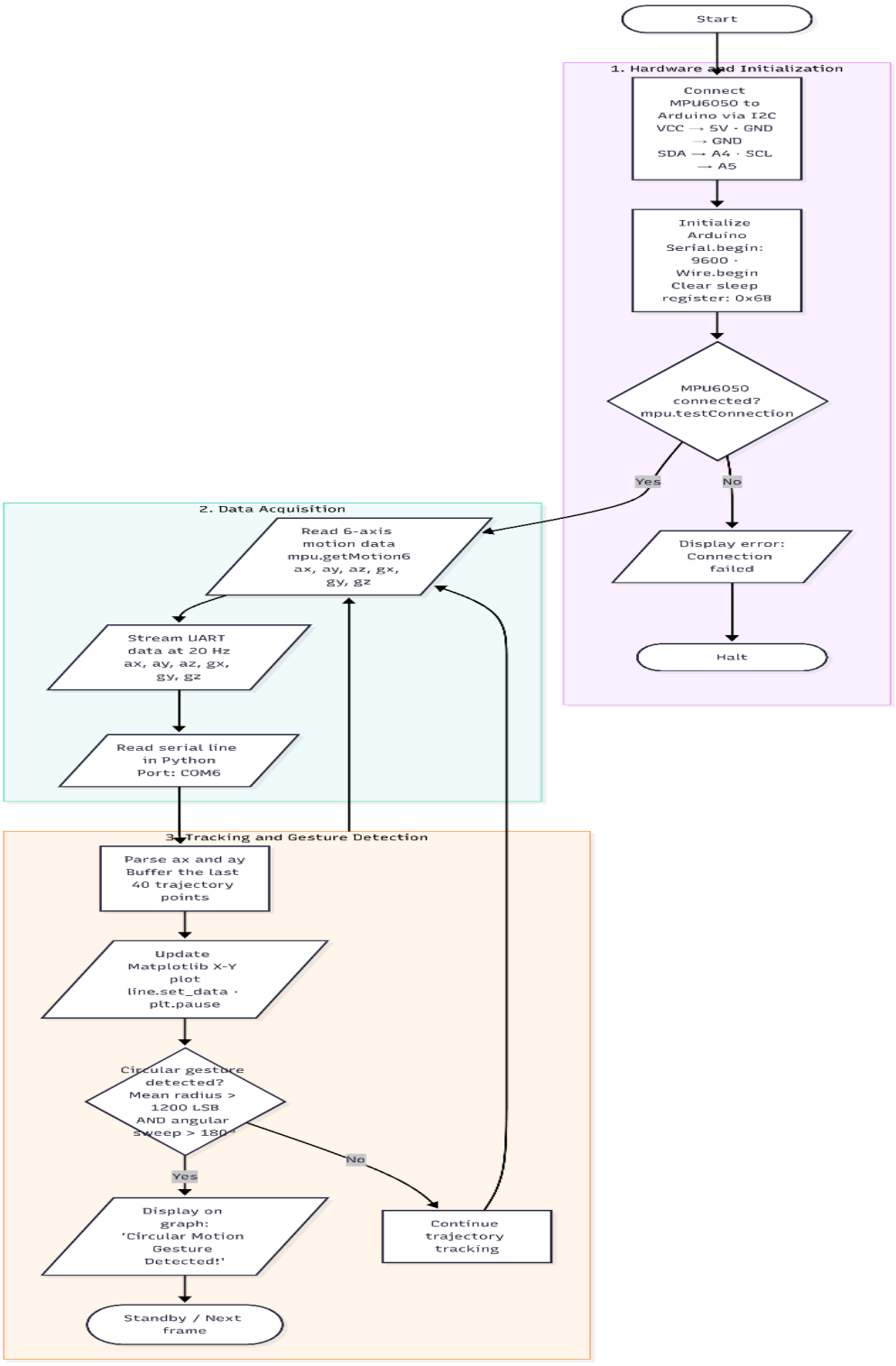


Figure 1.2 (Logic Flow Diagram for Part 1)

Complete Arduino C++ Firmware using Arduino

```
   #include <Wire.h>
#include <MPU6050.h>

MPU6050 mpu;

unsigned long lastSampleMs = 0;
const unsigned long SAMPLE_INTERVAL_MS = 50;

void setup() {
  Serial.begin(9600);
  Wire.begin();
  mpu.initialize();

  Wire.beginTransmission(0x68);
  Wire.write(0x6B);
  Wire.write(0);
  Wire.endTransmission(true);

  if (!mpu.testConnection()) {
    Serial.println("ERR: MPU6050 connection failed!");
    while (1);
  }
}

void loop() {
  unsigned long now = millis();
  if (now - lastSampleMs >= SAMPLE_INTERVAL_MS) {
    lastSampleMs = now;

    int16_t ax, ay, az, gx, gy, gz;
    mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);

    Serial.print(ax); Serial.print(",");
    Serial.print(ay); Serial.print(",");
    Serial.print(az); Serial.print(",");
    Serial.print(gx); Serial.print(",");
    Serial.print(gy); Serial.print(",");
    Serial.println(gz);
  }
}
```

Complete Python Host Script using Spyder (Anaconda 3):

```
import time
from collections import deque

import numpy as np
import serial
import matplotlib.pyplot as plt

SERIAL_PORT = '/dev/cu.usbmodemFX2348N1' #Using macbook IOS
BAUD_RATE = 9600
MAX_POINTS = 35

ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
time.sleep(2)
ser.reset_input_buffer()

x_vals = deque(maxlen=MAX_POINTS)
y_vals = deque(maxlen=MAX_POINTS)

plt.ion()
fig, ax = plt.subplots(figsize=(7, 7))
ax.set_xlim(-20000, 20000)
ax.set_ylim(-20000, 20000)
ax.set_title("Task 1: Real-Time MPU6050 Motion Tracking")
ax.set_xlabel("Acceleration X (raw LSB)")
ax.set_ylabel("Acceleration Y (raw LSB)")
ax.grid(True, linestyle='--', alpha=0.6)

line, = ax.plot([], [], 'ro-', linewidth=2, markersize=5, label='Hand
path')
center_dot, = ax.plot([], [], 'bx', markersize=10, label='Circle centre')
status_box = ax.text(-18500, 17500, "Status: Collecting data...",
                     fontsize=11, fontweight='bold',
                     bbox=dict(boxstyle='round', facecolor='wheat',
alpha=0.8))
ax.legend(loc='lower right')

def check_circular_gesture(x_arr, y_arr):
    if len(x_arr) < 16:
        return False, 0, 0

    x = np.array(x_arr, dtype=float)
    y = np.array(y_arr, dtype=float)

    cx, cy = np.mean(x), np.mean(y)

    span_x = np.max(x) - np.min(x)
    span_y = np.max(y) - np.min(y)
    if span_x < 6000 or span_y < 6000:
        return False, cx, cy
```

```
        dx = x - cx
        dy = y - cy
        q1 = np.any((dx > 0) & (dy > 0))
        q2 = np.any((dx < 0) & (dy > 0))
        q3 = np.any((dx < 0) & (dy < 0))
        q4 = np.any((dx > 0) & (dy < 0))
        if not (q1 and q2 and q3 and q4):
            return False, cx, cy

        radii = np.sqrt(dx ** 2 + dy ** 2)
        if (np.std(radii) / np.mean(radii)) >= 0.45:
            return False, cx, cy

        cross = dx[:-1] * dy[1:] - dy[:-1] * dx[1:]
        ccw = np.sum(cross > 0)
        cw = np.sum(cross < 0)
        if max(ccw, cw) / max(ccw + cw, 1) < 0.7:
            return False, cx, cy

        return True, cx, cy

    print("Tracking started. Move the sensor in a circle. Press Ctrl+C to
exit.")

    try:
        while plt.fignum_exists(fig.number):
            try:
                raw_line = ser.readline().decode('utf-8',
errors='ignore').strip()
                if not raw_line:
                    continue
                values = raw_line.split(',')

                if len(values) < 3:
                    continue

                x_vals.append(int(values[0]))
                y_vals.append(int(values[1]))

                if len(x_vals) >= 3:
                    kernel = np.ones(3) / 3
                    smooth_x = np.convolve(np.array(x_vals), kernel,
mode='valid')
                    smooth_y = np.convolve(np.array(y_vals), kernel,
mode='valid')
                else:
                    smooth_x, smooth_y = np.array(x_vals), np.array(y_vals)

                line.set_data(smooth_x, smooth_y)

                detected, cx, cy = check_circular_gesture(smooth_x, smooth_y)
                if detected:
                    center_dot.set_data([cx], [cy])
```

```
                    status_box.set_text("Status: CIRCULAR MOTION DETECTED!")
                    status_box.set_bbox(dict(boxstyle='round',
facecolor='lightgreen', alpha=0.9))
                else:
                    center_dot.set_data([], [])
                    status_box.set_text("Status: Tracking motion...")
                    status_box.set_bbox(dict(boxstyle='round',
facecolor='wheat', alpha=0.8))

                plt.draw()
                plt.pause(0.02)

            except ValueError:

                continue

    except KeyboardInterrupt:
       print("\nProgram terminated by user.")
    finally:
       ser.close()
       plt.close('all')
       print("Serial connection closed.")
```

1.4. DATA COLLECTION AND ANALYSIS

The program was configured to collect motion data from the MPU6050 sensor as it was moved by hand.. The Arduino read the accelerometer and gyroscope values every 50 ms and sent them to Python through serial communication at a baud rate of 9600 bps. Each line of data contained six values (ax, ay, az, gx, gy, gz). Only the X-axis and Y-axis acceleration values (ax and ay) were used for plotting and gesture detection. The data acquisition settings used in the program are summarised in Table 1.1.

Parameter	Value
Sampling interval	50 ms
Sampling rate	20 Hz
Baud rate	9600 bps
Data per line	6 values (ax, ay, az, gx, gy, gz)
Values used for analysis	ax and ay
Number of points displayed	35 (latest readings only)
Time window shown on graph	About 1.7s
Noise filter	3-point moving average
Graph range	±20 000 LSB on both axes

Table 1.1: Data Acquisition Settings Based on the Program

The values received from the MPU6050 are raw digital readings (LSB). At the default range of ±2 g, they can be converted into acceleration using the following equation:

$$a = \frac{Rawvalue}{16384} \times 9.81\ m/s^2$$

Using this equation, a raw value of 16 384 equals 1 g (9.81 m/s²). The graph limit of ±20 000

LSB corresponds to about ±1.22 g (±11.98 m/s²). The minimum movement span of 6 000 LSB required by the program corresponds to about 0.37 g (3.59 m/s²). Table 1.2 shows these conversions.

Raw Value (LSB)	**Acceleration (g)**	**Acceleration (m/s²)**	**Meaning in the Program**
6 000	0.37	3.59	Minimum movement span on each axis
16 384	1.00	9.81	Reading equal to 1 g (gravity)
20 000	1.22	11.98	Graph axis limit
32 767	2.00	19.62	Maximum sensor reading at ±2 g

Table 1.2: Conversion of Raw Sensor Values to Acceleration

Before the motion was analysed, a 3-point moving average was applied to the latest readings. Each plotted point is the average of three consecutive readings, which smooths out sudden spikes caused by sensor noise and small hand shakes. The program then calculated the centre of the motion path by taking the average of all X and Y values. It measured the distance of each point from this centre (radius) and checked the direction of movement between consecutive points. A gesture was accepted as circular only when all of the conditions in Table 1.3 were satisfied.

Condition	Requirement in the Program	Purpose
Enough data	At least 16 points	Ensures enough readings to judge the shape
Movement size	Span of X and Y each ≥ 6 000 LSB	Rejects small movements and noise
Full coverage	Points found in all four quadrants around the centre	Ensures the motion goes all the way around
Round shape	Standard deviation of radius ÷ mean radius < 0.45	Ensures the distance from the centre stays fairly constant
Same direction	At least 70% of points rotate in one direction	Rejects back-and-forth or random motion

Table 1.3: Circular Motion Detection Conditions Based on the Program

When all five conditions were met, the program marked the centre of the circle with a blue "x" and changed the status box to "CIRCULAR MOTION DETECTED!" in green. If any condition failed, the status remained "Tracking motion..." and the program continued reading new data.

1.5. RESULTS AND DISCUSSION

The Real-Time Motion Tracking experiment was designed to demonstrate how motion can be measured using an inertial sensor and processed on a computer through serial communication. The main objective was to capture hand movement using the MPU6050 sensor, display the motion path on a live X–Y graph in Python, and automatically detect when a circular hand gesture was performed.

The MPU6050 sensor was connected to the Arduino Uno through the I²C bus, with SDA connected to pin A4 and SCL connected to pin A5. The Arduino read the accelerometer and gyroscope values every 50 ms and sent them to the computer through USB serial communication at 9600 baud. On the computer, the Python script received the data, kept the latest 35 readings, smoothed them using a 3-point moving average, and plotted the X and Y acceleration values against each other in real time.

According to the programmed operating principle, a circular hand movement causes the X and Y acceleration values to rise and fall continuously, producing a loop-shaped path on the graph. The program calculated the centre of this path and checked five conditions:

1. Enough readings were collected.
2. The movement was large enough.
3. The points surrounded the centre in all four directions.
4. The distance from the centre was reasonably consistent.
5. Most points rotated in the same direction.

When all conditions were met, the status box changed to “CIRCULAR MOTION DETECTED!” and the circle centre was marked with a blue “x”. Otherwise, the status remained “Tracking motion...”.

Two movements were tested to evaluate the system: a circular hand motion and a non-circular movement. The circular motion was successfully detected. The non-circular movement was correctly rejected, as shown in Figures 1.3 to 1.5.

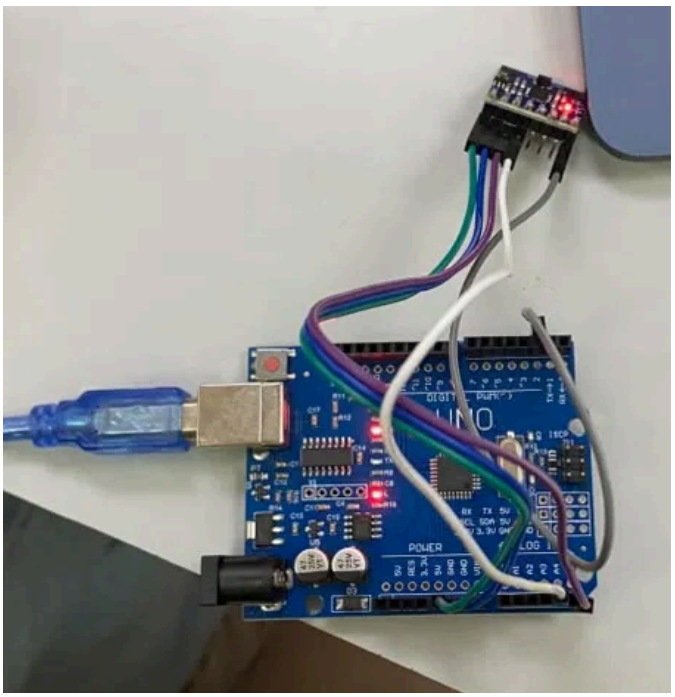


Figure 1.3: Physical hardware setup of the MPU6050 sensor connected to the Arduino Uno

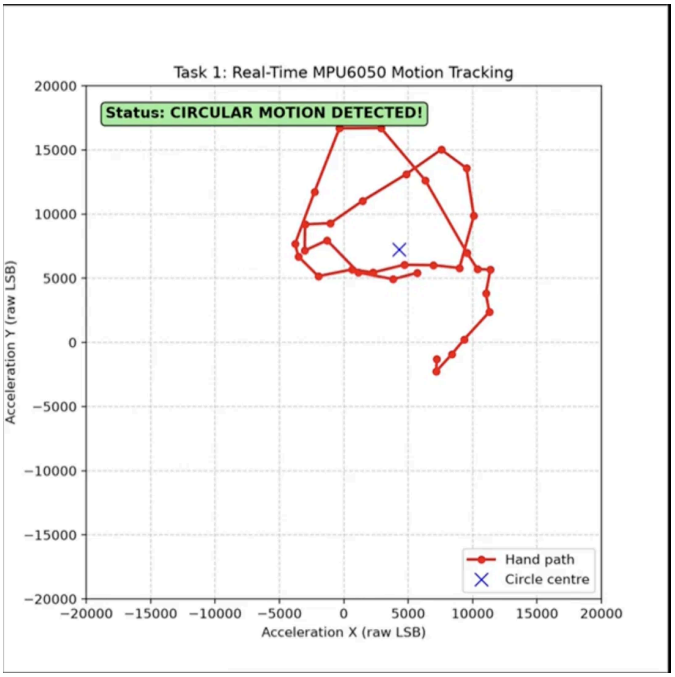


Figure 1.4: Real-time X–Y motion trajectory during a circular hand gesture

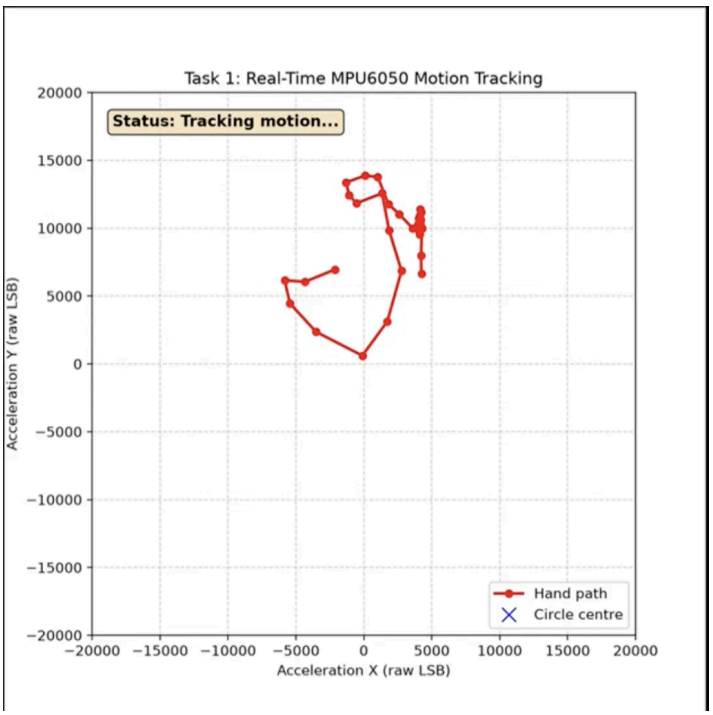


Figure 1.5: Real-time X–Y motion trajectory for a movement not classified as circular

Discussion of Errors and Technical Limitations

Several hardware and software factors may influence the performance of the real-time motion tracking and circular gesture detection system.

1. **Gravity effect on accelerometer readings**

The MPU6050 accelerometer measures gravity together with the hand motion. If the sensor is tilted during the gesture, part of the gravity reading appears on the X and Y axes. This shifts the plotted path away from the centre of the graph. This issue can be reduced by keeping the sensor level during the gesture, or by combining the accelerometer and gyroscope data to remove the effect of gravity.

2. **Sensor noise and hand shake**

Small hand shakes and sensor noise cause the plotted points to scatter, especially when the sensor is held still. The 3-point moving average reduces this noise but cannot remove it completely. This issue can be reduced by using a stronger filter, such as a longer moving average or the MPU6050's built-in low-pass filter.

3. **Low sampling rate**

The sensor was sampled at 20 Hz, so only a limited number of points are captured for each circle. As a result, the plotted path looks uneven and polygon-like rather than smooth. The 9600 baud serial speed also limits how fast data can be sent. This issue can be reduced by increasing the baud rate, which would allow a faster sampling rate and a smoother path.

4. Gesture speed and fixed detection settings

Only the latest 35 readings (about 1.7 s) are checked, so a circle performed too slowly may not be detected. The detection thresholds are also fixed and may not suit every user or gesture size. This issue can be reduced by adjusting the detection window and thresholds based on repeated testing with different users.

1.6. CONCLUSION

In conclusion, the objectives of Part 1 of this experiment were successfully achieved through the implementation of a real-time motion tracking system using the MPU6050 sensor, Arduino Uno, and Python. The experiment demonstrated how accelerometer data could be collected and transmitted through serial communication at a baud rate of 9600 bps before being processed and visualized using Python. Based on the results obtained, the system was able to display hand movements on a real-time X–Y graph and successfully identify circular motion while rejecting non-circular movements. The implementation of a 3-point moving average filter also helped reduce sensor noise and improve the clarity of the plotted motion path.

However, several limitations were observed during the experiment, including the influence of gravity on accelerometer readings, sensor noise, and the limited sampling rate, which affected the smoothness and accuracy of the motion trajectory. The detection performance was also influenced by the speed and consistency of hand movements. These limitations could be minimized by improving the filtering techniques, increasing the sampling rate, and adjusting the motion detection parameters. Overall, this experiment provided a better understanding of sensor interfacing, serial communication, real-time data processing, and motion detection. The knowledge and techniques gained from this experiment also serve as an important foundation for integrating motion verification into the RFID-based access control system in Part 2.

PART 2: SERVO MOTOR CONTROLLED WITH POTENTIOMETER

2.0. INTRODUCTION AND THEORY

This experiment focuses on developing a smart access control system using an Arduino, USB RFID reader, MPU6050 motion sensor, servo motor, and LED indicators. The system combines RFID identification and motion detection to improve access security. The primary objective of this experiment is to design and evaluate a multi-factor authentication mechanism by establishing bidirectional serial communication between an Arduino microcontroller and a Python host application. The USB RFID reader functions as a keyboard input device, allowing the card's Unique Identifier (UID) to be read directly by Python without processing it through Arduino. Python verifies the card's UID and analyzes motion data from the MPU6050 sensor to detect the required circular hand gesture. Once both conditions are satisfied, Python sends a command to Arduino to unlock the servo motor and activate the green LED. Otherwise, access is denied, the servo remains locked, and the red LED turns on. This experiment demonstrates the integration of USB input devices, motion sensing, serial communication, and actuator control in an automated access control system.

Radio Frequency Identification (RFID) is a wireless identification technology that uses radio frequency signals to identify RFID cards or tags. In this experiment, a USB RFID reader is connected directly to the computer and operates as a Human Interface Device (HID), functioning similarly to a keyboard. Specifically, the objective of utilizing this configuration is to streamline credential acquisition at the host level while offloading real-time motion acquisition and actuator driving to the embedded microcontroller. When a card is scanned, its UID is entered directly into the Python application as keyboard input. Python compares the received UID with a predefined list of authorized cards to determine whether the user can proceed to motion verification. The Arduino is not involved in processing RFID data and is only responsible for handling the connected sensors and controlling the servo motor and LED indicators based on commands received from Python.

2.1. MATERIALS AND EQUIPMENTS

1. Arduino Uno board
2. MPU6050 6-Axis IMU Sensor Module
    a. VCC connected to 5V Power Rail
    b. GND connected to GND Rail
    c. SDA connected to Analog Pin A4
    d. SCL connected to Analog Pin A5
    e. INT connected to Digital Pin D2
3. RFID Module / Card Reader (USB Desktop RFID Reader)
    a. Connected directly to the host computer's USB Port (Keyboard emulation HID mode)



4. RFID Cards
5. Servo Motor
    a. Signal wire (Orange) connected to Digital Pin D9
    b. Power wire (Red) connected to 5V Power Rail
    c. Ground wire (Brown) connected to GND Rail
6. Green LED (Access Granted Indicator)
    a. Anode (+) connected to Digital Pin D11 via a current-limiting resistor
    b. Cathode (-) connected to GND Rail
7. Red LED (Access Denied Indicator)
    a. Anode (+) connected to Digital Pin D10 via a current-limiting resistor
    b. Cathode (-) connected to GND Rail
8. 220 Ω resistors (2 units for LED current regulation)
9. Jumper wires (male-to-male and male-to-female jumpers)
10. Breadboard
11. Personal Computer with Arduino IDE and Python Environment

2.2. EXPERIMENTAL SETUP

Circuit Wiring & Hardware Connection Instructions (as in Figure 2.1)**:**

1. **Shared Power and Ground Distribution:**
    a. **Arduino 5V Pin →** the breadboard **Positive Power Rail (+)**
    b. **Arduino GND Pin →** the breadboard **Negative Ground Rail (−)**
2. **MPU6050 Motion Sensor (I2C Interface):**
    - **VCC Pin → 5V Power Rail (+)**
    - **GND Pin → Ground Rail (−)**
    - **SDA Pin → Arduino Analog Input Pin A4**
    - **SCL Pin → Arduino Analog Input Pin A5**
    - **INT Pin → Arduino Digital Input Pin D2**
3. **RFID Authentication Reader Setup:**
    - **USB RFID Reader cable →** an open **USB Port** on the host PC
4. **Servo Lock Mechanism Wiring:**
    - **Signal Wire (Orange/Yellow) → Digital PWM Pin D9**
    - **Power Wire (Red) → 5V Power Rail (+)**
    - **Ground Wire (Brown/Black) → Ground Rail (−)**
5. **Visual Feedback Indicator LEDs (Current-Limited Outputs):**
    - **Green LED (Access Granted):**
        i. **Anode (+) → 220 Ω resistor → Digital Pin D11**
        ii. **Cathode (−) → Ground Rail (−)**
    - **Red LED (Access Denied):**
        i. **Anode (+) → 220 Ω resistor → Digital Pin D10**
        ii. **Cathode (−) → Ground Rail (−)**
6. **Host Computer Interface:**
    - Connect the **Arduino Uno** via USB cable to the computer running the **Python** host controller application.

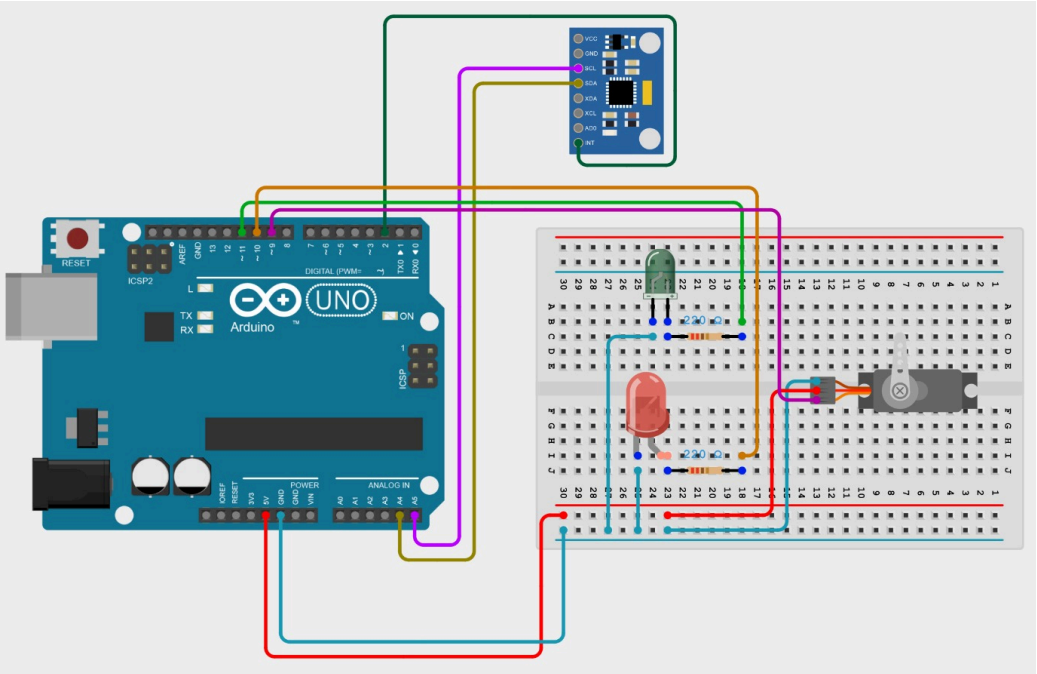


Figure 2.2: The hardware connection of the system

2.3. METHODOLOGY AND PROGRAMMING LOGIC

Procedural Steps:

1. Assemble the hardware components following the setup in Section 2.2**.**
2. **Connect both the Arduino Uno and the USB desktop RFID reader** to available USB ports on the host computer.
3. **Open the Arduino IDE**, load the integrated security firmware sketch (W4T2.ino), compile, and upload it to the Arduino Uno.
4. **Verify initial hardware status**: Upon powering on, the system enters a default **LOCKED** state with the servo motor positioned at 90°, the **Red LED ON**, and the **Green LED OFF**.
5. **Capture card UID**: Scan an RFID card using the desktop reader into a text editor (e.g., Notepad) to read its unique UID string (e.g., "0013004466").
6. **Configure Python authorization**: Paste the valid RFID UID string into the authorized_cards list inside the Python host script (W4T2.py).
7. **Execute the Python host script** to launch the background RFID keyboard listener thread, establish UART serial communication with the Arduino, and display the real-time Matplotlib dashboard.
8. **Execute the multi-factor authentication test sequence**:
    - **Step A (RFID Authentication)**: Tap an RFID card on the reader. If unauthorized, Python transmits byte 'D', holding the system in a **LOCKED** state.
    - **Step B (Gesture Recording & Circular Motion Detection)**: If the card UID is authorized, Python prompts the user to perform a circular motion gesture with the MPU6050 sensor. Python records the X-Y accelerometer trajectory and plots it live on the Matplotlib graph while analyzing whether a valid circular pattern is detected.
    - **Step C (Mathematical Evaluation & Actuation)**: Python evaluates gesture radius ($R > 1200LSB$) and total angular sweep ($\Delta\theta > 180^{\circ}$). If circular motion is verified, Python sends byte b'A' to rotate the servo to 180° (**UNLOCKED**) and turn ON the **Green LED**. If unverified or invalid, byte b'D' is sent to maintain the **LOCKED** state (Red LED ON, Servo 90°).

9. **Safely terminate the system** by pressing Ctrl + C or typing 'Q', ensuring Python sends a final lock command (b'D') and releases the serial port handle.

Logic Flow Diagrams (Figure 2.2):

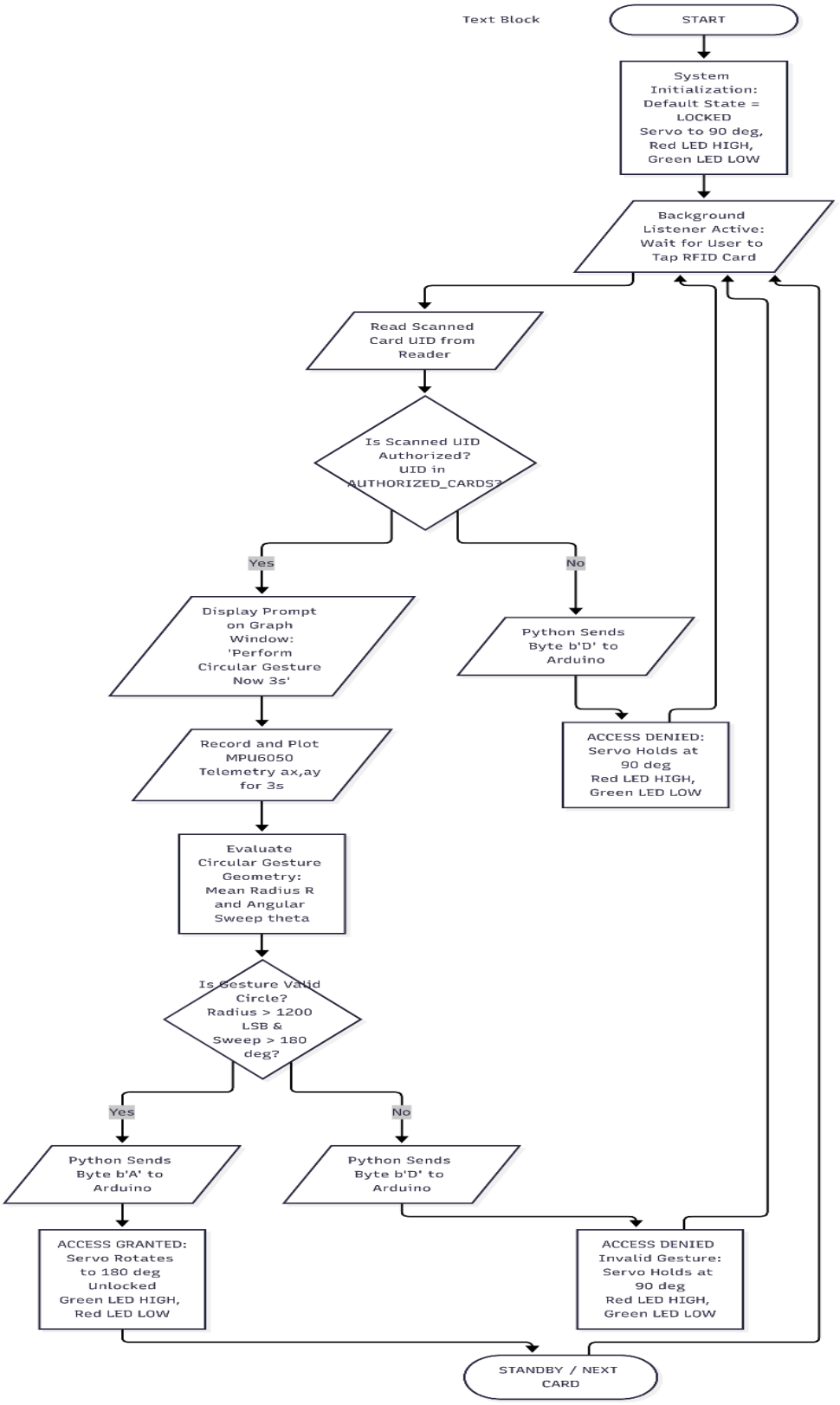


Figure 2.2 (Logic Flow Diagram for Part 2)

Complete Arduino C++ Control Firmware using Arduino IDE:

```
#include <Wire.h>
#include <MPU6050.h>
#include <Servo.h>

MPU6050 mpu;
Servo servo;
int servoPosition = 90;

void setup() {
  servo.attach(9);
  servo.write(servoPosition); // Start in locked position (90 deg)

  pinMode(11, OUTPUT);  // Green LED
  pinMode(10, OUTPUT);  // Red LED

  digitalWrite(11, LOW);
  digitalWrite(10, HIGH);

  Serial.begin(9600);
  Wire.begin();
  mpu.initialize();
}

void loop() {
    if (Serial.available() > 0) {
    char command = Serial.read();
    if (command == 'A') {
      servo.write(180);       // Unlock servo
      digitalWrite(11, HIGH); // Green LED ON
      digitalWrite(10, LOW);  // Red LED OFF
    } else if (command == 'D') {
      servo.write(90);        // Lock servo
      digitalWrite(11, LOW);  // Green LED OFF
      digitalWrite(10, HIGH); // Red LED ON
    }
  }
  int16_t ax, ay, az, gx, gy, gz;
  mpu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
  Serial.print(ax); Serial.print(",");
  Serial.print(ay); Serial.print(",");
  Serial.print(az); Serial.print(",");
```

```
    Serial.print(gx); Serial.print(",");
    Serial.print(gy); Serial.print(",");
    Serial.println(gz);

    delay(50);
  }
```

Complete Python Host Script using Spyder (Anaconda 3):

```
import matplotlib
matplotlib.use('TkAgg')

import serial
import time
import threading
import numpy as np
import matplotlib.pyplot as plt

SERIAL_PORT = 'COM6'
BAUD_RATE = 9600
authorized_cards = ["0013004466"]
scanned_uid = ""

def get_rfid():
    global scanned_uid
    while True:
        scanned_uid = input("\nTap RFID card (or 'q' to quit):
").strip().upper()

threading.Thread(target=get_rfid, daemon=True).start()

ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=0.1)
time.sleep(2)
ser.reset_input_buffer()
ser.write(b'D')

plt.ion()
fig, ax = plt.subplots()
x_vals, y_vals = [], []

def check_circle(x_list, y_list):
    if len(x_list) < 10:
        return False
    dx = np.array(x_list) - np.mean(x_list)
    dy = np.array(y_list) - np.mean(y_list)
    radius = np.mean(np.hypot(dx, dy))
    angles = np.unwrap(np.arctan2(dy, dx))
    total_angle = np.degrees(np.ptp(angles))
    print(f"Radius: {radius:.0f}, Angle: {total_angle:.0f} deg")
    return radius > 1000 and total_angle > 160
```

```
    try:
        while True:
            line = ser.readline().decode('utf-8', errors='ignore').strip()
            if line:
                values = line.split(',')
                if len(values) >= 2:
                    try:
                        ax_val = int(values[0])
                        ay_val = int(values[1])
                        x_vals.append(ax_val)
                        y_vals.append(ay_val)
                        x_vals = x_vals[-30:]
                        y_vals = y_vals[-30:]

                        ax.clear()
                        ax.plot(x_vals, y_vals, 'ro-')
                        ax.set_xlim(-40000, 40000)
                        ax.set_ylim(-40000, 40000)
                        ax.set_title("MPU6050 Real-Time Motion")
                        ax.grid(True)
                    except ValueError:
                        pass

            plt.pause(0.01)

            if scanned_uid:
                uid = scanned_uid
                scanned_uid = ""

                if uid == 'Q':
                    break
                elif uid in authorized_cards:
                    print("RFID authorized! Perform circular motion for 3
seconds...")
                    x_vals, y_vals = [], []
                    ser.reset_input_buffer()
                    start_time = time.time()

                    while time.time() - start_time < 3.0:
                        line = ser.readline().decode('utf-8',
errors='ignore').strip()
                        if line:
                            values = line.split(',')
                            if len(values) >= 2:
                                try:
                                    x_vals.append(int(values[0]))
                                    y_vals.append(int(values[1]))
                                    ax.clear()
                                    ax.plot(x_vals, y_vals, 'ro-')
                                    ax.set_xlim(-40000, 40000)
                                    ax.set_ylim(-40000, 40000)
                                    ax.set_title("Recording Circular Motion...")
                                    ax.grid(True)
```

```
                        except ValueError:
                            pass
                    plt.pause(0.01)

                if check_circle(x_vals, y_vals):
                    print("Motion verified. Access granted.")
                    ser.write(b'A')
                else:
                    print("Motion invalid. Access denied.")
                    ser.write(b'D')
            else:
                print("UID unauthorized. Access denied.")
                ser.write(b'D')

except KeyboardInterrupt:
    print("Program terminated.")
finally:
    ser.write(b'D')
    ser.close()
    plt.ioff()
    plt.close('all')
```

2.4. DATA COLLECTION AND ANALYSIS

During the experiment, data was collected using the USB RFID reader and MPU6050 motion sensor. The RFID reader acted as a keyboard input device, allowing Python to receive the card's Unique Identifier (UID) directly. The authorized UID was set to 0013004466. Meanwhile, the MPU6050 sensor measured acceleration and angular velocity along the X, Y, and Z axes. The Arduino transmitted these readings to Python through serial communication at a baud rate of 9600 bps, with a sampling delay of 50 ms.

The motion data was plotted in real time using Matplotlib to visualize hand movement along the X and Y axes. Once the system detected an authorized RFID card, it recorded motion data for three seconds. Python analyzed the movement by calculating the average radius and total angular variation using NumPy. A circular gesture was accepted when the calculated radius exceeded 1000, and the angular variation exceeded 160°. The system used the following conditions to determine access permission.

RFID STATUS	MOTION DETECTION	SERVO POSITION	LED INDICATOR
Authorized	Radius > 1000 AND angle > 160°	180° (**Unlock**)	**Green** ON
Authorized	Motion criteria not met	90° (**Lock**)	**Red** ON
Not Authorized	Not evaluated	90° (**Lock**)	**Red** ON

Table 1.1: Access Control Conditions Based on the Program

When both RFID identification and motion verification succeeded, Python sent command A to the Arduino, causing the servo motor to rotate to 180° and activating the green LED. Otherwise, Python sent the command D, keeping the servo at 90° and activating the red LED.

Based on the programmed conditions, the system was designed to grant access only when both verification requirements were satisfied. The real-time motion graph also allowed the movement pattern to be observed during the verification process.

2.5. RESULTS AND DISCUSSION

This part focused on developing a smart access control system by integrating an RFID reader, MPU6050 motion sensor, Arduino Uno, servo motor, and LED indicators. The main objective was to demonstrate how RFID authentication and motion detection could be combined to control an access system through serial communication between Arduino and Python.

In this experiment, the USB RFID reader was connected directly to the computer and used to identify RFID cards based on their Unique Identifier (UID). The authorized UID was programmed as 0013004466. Meanwhile, the MPU6050 sensor collected acceleration data, which was transmitted from the Arduino to Python at a baud rate of 9600 bps. The received X-axis and Y-axis acceleration values were plotted using Matplotlib to visualize the hand movements in real time.

The system was programmed to operate in a locked condition initially, with the servo motor positioned at 90°, the red LED turned ON, and the green LED turned OFF. When an authorized RFID card was scanned, the Python program prompted the user to perform a circular hand gesture within three seconds. The collected motion data was then analyzed based on the average radius and total angular variation. According to the implemented Python code, a circular gesture was considered valid when the average radius exceeded 1000 LSB and the angular variation exceeded 160°.

When both the RFID authentication and circular motion requirements were satisfied, Python transmitted the command 'A' to the Arduino. This command caused the servo motor to rotate from 90° to 180°, representing the unlocked position, while the green LED was activated to indicate that access had been granted. However, if an unauthorized RFID card was scanned or the required circular motion was not detected, Python transmitted the command 'D'. Consequently, the servo motor remained at 90°, and the red LED indicated that access was denied. The programmed responses of the access control system under different conditions are summarized in Table 2.1.

RFID Status	Motion Detection	Servo Position	LED Indicator	Access Status
Authorized	Valid motion	180°	Green ON	Granted
Authorized	Invalid motion	90°	Red ON	Denied
Unauthorized	Not evaluated	90°	Red ON	Denied

Table 2.1: Access Control System Responses Based on the Program

The system design demonstrates the importance of combining multiple verification methods to improve access control. Unlike a conventional RFID system that relies only on card identification, this system requires an additional circular hand gesture before access can be granted. This provides an extra verification step and demonstrates the application of sensor integration in an automated security system.

However, several factors could affect the performance and reliability of the system. The MPU6050 sensor may produce fluctuating acceleration readings due to hand shaking, sensor noise, and changes in sensor orientation. These variations could affect the calculated radius and angular variation, potentially causing an incorrect motion detection result. Furthermore, the three-second recording period may not be sufficient for users who perform the circular gesture slowly. Serial communication delays and unstable wiring connections could also influence the response of the servo motor and LED indicators.

To improve the system, filtering techniques such as a moving average or low-pass filter could be implemented to reduce sensor noise. The motion detection thresholds and recording duration could also be adjusted to accommodate different gesture speeds and movement patterns. Additionally, ensuring stable power connections and proper serial communication would help improve the overall reliability of the system.

2.6. CONCLUSION

In conclusion, Part 2 of this experiment demonstrated the integration of an RFID reader, MPU6050 motion sensor, Arduino Uno, servo motor, and LED indicators to develop a smart access control system using serial communication between Arduino and Python. The experiment showed how RFID authentication and circular hand gesture detection could be combined to provide an additional level of access verification.

Based on the results and programmed operating conditions, the system was designed to grant access only when both authentication requirements were satisfied. An authorized RFID card allowed the user to proceed with motion verification, where the MPU6050 sensor recorded hand movements for three seconds. The Python program then analyzed the acceleration data by calculating the average radius and angular variation to determine whether a valid circular gesture had been performed. When both conditions were met, the Arduino received the command to rotate the servo motor from 90° to 180°, while the green LED indicated access granted. Otherwise, the servo remained at 90°, and the red LED indicated access denied.

However, several limitations were identified, including sensor noise, variations in hand movements, and possible delays in serial communication. The fixed three-second recording period could also affect gesture detection, particularly for slower movements. These limitations could be reduced by implementing improved filtering techniques, adjusting the gesture detection thresholds, and ensuring stable hardware connections.

Overall, this experiment provided a better understanding of sensor integration, bidirectional serial communication, real-time motion processing, and actuator control. It also demonstrated the potential application of combining RFID identification and gesture verification in automated access control systems.

RECOMMENDATIONS

Based on the results and observations from both tasks, several improvements are recommended to enhance the performance, accuracy, and reliability of the motion-activated security access system integrating Arduino and Python.

Improve Serial Communication and Port Stability: Ensure matching COM ports and a 9600 baud rate in both environments. The Arduino Serial Monitor must be closed before running Python to prevent port access conflicts, and a brief two-second startup delay should be included to let the serial connection stabilize.

Enhance Sensor Signal Quality: To minimize accelerometer noise and hand jitter, software filtering such as a moving average, low-pass, or Kalman filter should be applied. Adding pull-up resistors on the I2C lines (A4 and A5) and a bypass capacitor across the MPU6050 power pins will also help maintain bus stability during movement.

Improve Servo Motor Power Decoupling: Powering the servo motor from an external regulated 5 V source with a common ground will prevent current spikes and voltage sags on the Arduino board. Adding a decoupling capacitor across the servo power rail further prevents sudden brownouts and microcontroller resets.

Upgrade Gesture Verification and Access Control: The circular gesture detection can be strengthened using machine learning or dynamic pattern matching to reduce false triggers. Replacing the hardcoded authorization list with a database will also enable secure user management and automated event logging.

Optimize Real-Time Visualization and Logging: Graph rendering can be made smoother by using faster, event-driven visualization libraries like PyQtGraph to eliminate interface lag. In addition, automatically saving telemetry data and access timestamps to a CSV file will facilitate post-experiment analysis.

Overall, these recommendations will improve communication stability, sensor fidelity, and motor responsiveness, supporting more reliable real-time tracking and access control implementations in mechatronic systems.

REFERENCES

1. Last Minute Engineers. (2018, July 30). What is RFID? How it works? Interface RC522 RFID module with Arduino. Last Minute Engineers; Last Minute Engineers. https://lastminuteengineers.com/how-rfid-works-rc522-arduino-tutorial/

2. Santos, R. (2016, March 23). MFRC522 RFID reader with Arduino tutorial | random nerd tutorials. Random Nerd Tutorials. https://randomnerdtutorials.com/security-access-using-mfrc522-rfid-reader-with-arduino/

3. Arduino time attendance system with RFID | random nerd tutorials. (2018, May 24). https://randomnerdtutorials.com/arduino-time-attendance-system-with-rfid/

4. Instructables. (2016, December 5). Arduino + MFRC522 RFID reader. Instructables. https://www.instructables.com/Arduino-MFRC522-RFID-READER/

5. Naosekpam, V., & Sharma, R. K. (2019). Machine learning in 3D space gesture recognition. Jurnal Kejuruteraan, 31(2), 243-248.

6. Jian, M. S., & Wu, J. S. (2013). RFID applications and challenges. Radio frequency identification from system to applications.

7. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in science & engineering, 9(3), 90-95.

APPENDICES

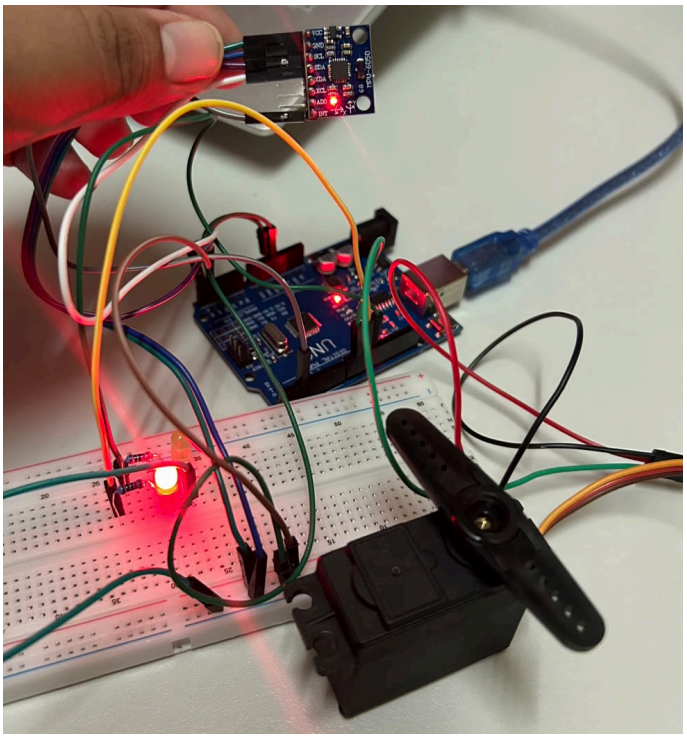


Figure A: The Hardware Connection for Task 2

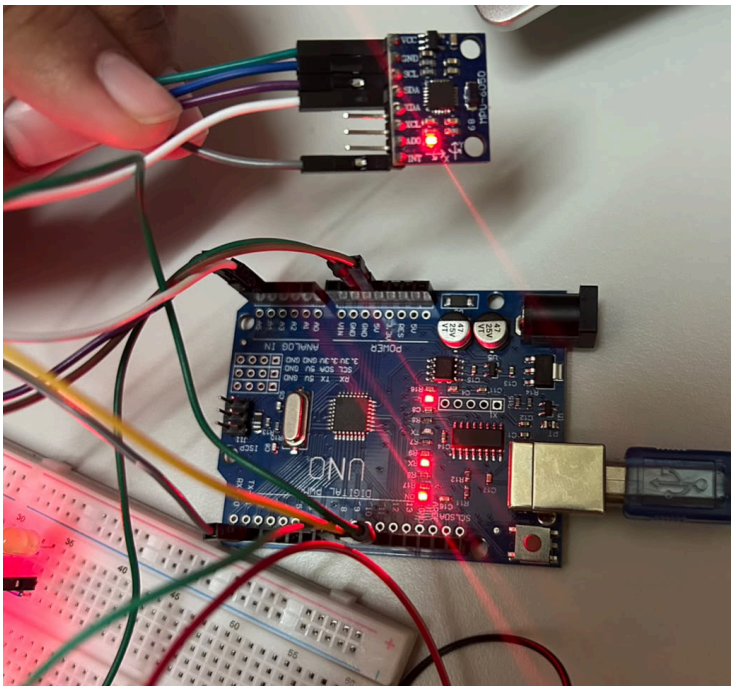


Figure B: The MPU6050 Connected to Arduino Uno for Task 2

ACKNOWLEDGEMENT

First and foremost, we would like to express our gratitude to our lecturer, Dr. Ahmad Imran bin Ibrahim, an Assistant Professor in the Department of Mechatronics Engineering, and our instructor for this experiment for providing guidance, instructions, and valuable explanations throughout this Serial Communication between Arduino and Python experiment, as well as actively supervising our laboratory session until the end of the class. The guidelines provided were helpful in assisting us in understanding the circuit connections and overcoming the difficulties encountered during the experiment.

We would also like to extend our sincere appreciation to the Department of Mechatronics Engineering, International Islamic University Malaysia (IIUM), for providing the experimental venue, complete materials and equipment, essential guidelines, and supportive laboratory staff who assisted us throughout our practical activities.

STUDENT'S DECLARATION

We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgement, and that the original work contained herein has not been undertaken or done by unspecified sources or persons. We hereby certify that this report has **not been done by only one individual and all of us have contributed to the report.** The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have **read and understand** the content of the total report and no further improvement on the reports is needed from any of the individual's contributors to the report. We therefore agreed unanimously that this report shall be submitted for **marking,** and this **final printed report** has been **verified by us**.

Name: AHMAD AIMAN BIN ABDUL HALIM Matric No.: 2417921 Signature: 	Read	✓
	Understand	✓
	Agree	✓

Name: AFIQ ASRAF BIN MOHD EZUAN'IZAM Matric No.: 2410541 Signature: 	Read	✓
	Understand	✓
	Agree	✓

Name: ‘AISYAH EL-’ILLIYYINA BT MOHD FAUZI Matric No.: 2319982 Signature: 	Read	✓
	Understand	✓
	Agree	✓

Name: AFIQ HUSAINI BIN MOHAMAD IZANI Matric No.: 2411757 Signature: 	Read	✓
	Understand	✓
	Agree	✓